

The project has a total of 48 endpoints in the main NestJS backend (Biconoirs_Backend_ClassOperation). Out of these, exactly 39 are Data Access endpoints (pure CRUD operations in /orders, /menu, /inventory, etc.) and 9 are Business Logic (BL) endpoints (e.g., /auth/register, /auth/login, /surveys/:surveyId/submit, /customers/:customerId/preferences, etc.). Additionally, the Supabase microservice contains 17 read-only endpoints.

Inconsistencies Analysis

1. Plural vs. Singular (-2): RESTful conventions require the use of plural nouns for collections, and the NestJS backend handles this correctly (e.g., /menu/dishes, /customers). However, the Supabase backend breaks consistency by using singular nouns: GET /menu/dish/:dishId and GET /customer/:customerId/orders.

```
@Controller("menu")
export class MenuController {
  constructor(private readonly menuService: MenuService) {}

  @Get("dishes")
  getDishes() {
    return this.menuService.getDishes();
  }

  @Get("dishes/:dishId")
  getDish(@Param("dishId") dishId: string) {
    return this.menuService.getDish(dishId);
  }

  @Post("dishes")
  createDish(@Body() dto: CreateDishDto) {
    return this.menuService.createDish(dto);
  }

  @Put("dishes/:dishId")
  updateDish(@Param("dishId") dishId: string, @Body() dto: Upd
    return this.menuService.updateDish(dishId, dto);
  }
}
```

2. Verbs in URIs (-1): In a strict REST architecture, URIs should be nouns. The endpoint POST /surveys/:surveyId/submit uses the verb "submit". (While the /auth endpoints also use verbs like login and register, these are generally tolerated as industry exceptions; however, "submit" is a clear deviation from RESTful design).

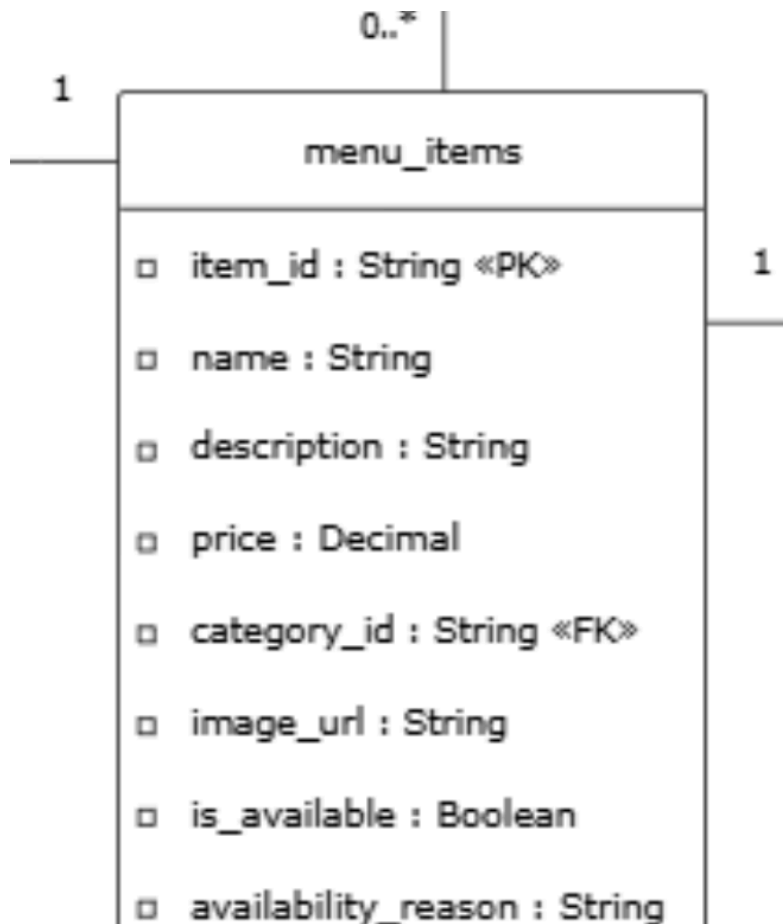
Penalty: -3 points

```

1  import { Body, Controller, Get, HttpStatusCode, Param, Post } from "
2  import { SurveysService } from "../surveys.service";
3  import { SubmitSurveyDto } from "../dto/submit-survey.dto";
4
5  @Controller("surveys")
6  export class SurveysController {
7      constructor(private readonly surveysService: SurveysService)
8
9      @Get()
10     findAll() {
11         return this.surveysService.findAll();
12     }
13
14     @Get(":surveyId")
15     findOne(@Param("surveyId") surveyId: string) {
16         return this.surveysService.findOne(surveyId);
17     }
18
19     @Post(":surveyId/submit")
20     @HttpCode(201)
21     submit(@Param("surveyId") surveyId: string, @Body() dto: Sub
22         return this.surveysService.submit(surveyId, dto);
23     }
24 }

```

3. URI Consistency (Features): While the majority of the API follows RESTful conventions, there are three notable consistency flaws. First, there is a pluralization conflict between microservices: Supabase functions improperly use singular nouns (e.g., GET /menu/dish/:dishId, GET /customer/:customerId/orders), breaking the standard set by the main NestJS API which correctly uses plural nouns (/menu/dishes, /customers). Second, the endpoint POST /surveys/:surveyId/submit violates strict REST design by incorporating a verb ("submit") in the URI rather than targeting a noun-based resource.
4. Use Case Diagram (UCD) Traceability: There is a noticeable gap between the system requirements modeled in the Use Case Diagram and the final API implementation. Three major use cases mapped to the Customer and Administrator actors completely lack corresponding endpoints in the API layer: "View Location and Contact", "View Audit Log (Read-Only)", and "View Financial Reports (History & PLPD)".
5. Class Diagram (CD) Consistency: The API's RESTful URIs fail to consistently map their nomenclature to the domain entities modeled in the Class Diagram. Specifically, the API exposes /customers instead of mapping to the users class, uses /menu/dishes instead of menu_items, and utilizes /cart instead of shopping_carts. Furthermore, the survey endpoint fails to map to the expected survey_responses entity model.



Suggested Final Score: 93.6/ 100

UCD (Use Case Diagram): 7 / 10

Justification: Based on the submitted Use Case Diagram, most use cases are correctly implemented in the backend. However, endpoints are missing for three important use cases shown in the diagram: "View Location and Contact," "View Audit Log," and "View Financial Reports." A penalty of -1 point is applied for each use case that cannot be traced to the API.

CD (Class Diagram): 9.6 / 10

Justification: Most routes are logically aligned with the design. However, there are four endpoints whose names do not map exactly to the entities defined in the Class Diagram (`users` → `/customers`, `menu_items` → `/menu/dishes`, `shopping_carts` → `/cart`, `survey_responses` → `.../submit`). Applying the -0.1 penalty for each inconsistency results in a score of 9.6.

REST: 17 / 20

Justification: The use of HTTP methods (GET, POST, PUT, PATCH, DELETE) is appropriate for nearly all operations. A deduction of 3 points is applied due to violations of RESTful

principles previously identified: the inconsistent use of singular nouns in the Supabase microservice (/dish/:dishId) compared to the plural naming convention used in NestJS, and the use of the verb "submit" in the survey URI.

ORM: 10 / 10

Justification: Excellent integration of Prisma ORM. The schema (schema.prisma) is well structured, the Prisma client is properly injected through a dedicated module (PrismaService), and database queries are handled asynchronously and cleanly within the service layer (e.g., `this.prisma.reservation.findMany()`).

Clean Code: 28 / 30

Justification: The code quality is highly professional. The project demonstrates excellent separation of responsibilities (Controllers vs. Services). It makes rigorous use of DTOs (Data Transfer Objects) for input validation and handles errors using NestJS native exceptions (e.g., `NotFoundException`).

Reason for the 2-point deduction: Minor technical code smells are present, such as hardcoded string values when parsing dates in the reservation service (`new Date(\1970-01-01T${dto.reservation_time}:00Z`)`) instead of using dedicated date/time libraries or reusable utility functions. Nevertheless, the overall structure is excellent and well designed.

```
return this.prisma.reservation.create({
  data: {
    reservationId: crypto.randomUUID(),
    userId,
    reservationDate: new Date(dto.reservation_date),
    reservationTime: new Date(`1970-01-01T${dto.reservation_time}:00Z`),
    partySize: dto.party_size,
    specialRequests: dto.special_requests,
    status: "pending",
  },
});
```



```

42     async update(userId: string, reservationId: string, dto: UpdateReservationDto) {
43         await this.requireUser(userId);
44
45         if (Object.keys(dto).length === 0) {
46             throw new BadRequestException("At least one field is required");
47         }
48
49         const existing = await this.prisma.reservation.findUnique({ where: { reservationId } })

```

Método	URI	It is working (y/n)
POST	/auth/register	y ?
POST	/auth/login	y ?
POST	/auth/refresh	n
POST	/auth/logout	y
GET	/auth/logout	y
POST	/auth/password/forgot	/y
POST	/auth/password/reset	-
PUT	/customers/:customerId	y
GET	/customers/:customerId/orders	y
PATCH	/customers/:customerId/preferences	y
GET	/menu/dishes	y
GET	/menu/dishes/:dishId	y
POST	/menu/dishes	y
PUT	/menu/dishes/:dishId	y
DELETE	/menu/dishes/:dishId	y
PATCH	/menu/dishes/:dishId/availability	y
POST	/menu/dishes/:dishId/ingredients	n
DELETE	/menu/dishes/:dishId/ingredients/:sku	

GET	/menu/categories	y
POST	/menu/categories	y
PUT	/menu/categories/:categoryId	y
GET	/menu/ingredients	y
GET	/orders	y
GET	/orders/:orderId	y
POST	/orders	y
PUT	/orders/:orderId	y
DELETE	/orders/:orderId	y
GET	/surveys	n 404
GET	/surveys/:surveyId	-
POST	/surveys/:surveyId	-
GET	/cart	y
POST	/cart/items	y
PATCH	/cart/items/:dishId	y
DELETE	/cart/items/:dishId	y
DELETE	/cart	y

HTTP  http://3.20.57.154:3000/ops/menu/dishes/dish_001  Save  Share 

GET  http://3.20.57.154:3000/ops/menu/dishes/dish_001  Send 

 Docs Params Authorization Headers 6 **Body** Scripts Tests Settings [Cookies](#)

none 

This request does not have a body

Body  200 OK • 249 ms • 618 B •  ...

 JSON  Preview  Visualize 

```
1 {
2   "itemId": "dish_001",
3   "name": "Tartar de Salmón y Aguacate",
4   "description": "Salmón fresco marinado en cítricos sobre cama de aguacate premium.",
5   "price": "12.5",
6   "categoryId": "cat_entradas",
7   "imageUrl": "https://biconoirsgourmet/images/tartar.jpg",
8   "isAvailable": true,
9   "availabilityReason": "Mantenimiento del equipo",
10  "createdAt": "2026-06-09T22:18:54.489Z"
11 }
```

HTTP  http://3.20.57.154:3000/ops/cart/items/dish_001  Save 

PATCH  http://3.20.57.154:3000/ops/cart/items/dish_001

 to favorites Params Authorization Headers 10 **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON   Schem

```
1 {
2   "quantity": 2
3 }
```

Body Cookies Headers 8 Test Results  200 OK • 362 ms • 82:

 JSON  Preview  Visualize 

```
1 {
2   "userId": "578a50a1-129c-4354-85e9-b09ac07b92c4",
3   "updatedAt": "2026-06-14T01:04:39.687Z",
4   "items": [
5     {
6       "userId": "578a50a1-129c-4354-85e9-b09ac07b92c4",
7       "itemId": "dish_001",
8       "quantity": 2,
9       "item": {
10        "itemId": "dish_001",
11        "name": "Tartar de Salmón y Aguacate",
12        "description": "Salmón fresco marinado en cítricos sobre cama de aguacate premium.",
13        "price": "15.99",
```

HTTP **http://3.20.57.154:3000/ops/cart** Save

Switch request type **http://3.20.57.154:3000/ops/cart**

Docs Params Authorization **Headers 7** Body Scripts Tests Settings

Headers 6 hidden

Key	Value	Description	Bulk Edit
☒ x-user-id	578a50a1-129c-4354-85e9-b09ac07b92c4		
Key	Value	Description	

Body Cookies Headers 8 Test Results 200 OK • 264 ms

{} JSON Preview Visualize

```
1 {
2   "userId": "578a50a1-129c-4354-85e9-b09ac07b92c4",
3   "updatedAt": "2026-06-14T00:42:44.659Z",
4   "items": [
5     {
6       "userId": "578a50a1-129c-4354-85e9-b09ac07b92c4",
7       "itemId": "dish_001",
8       "quantity": 4,
9       "item": {
10        "itemId": "dish_001",
11        "name": "Tartar de Salmón y Aguacate",
12        "description": "Salmón fresco marinado en cítricos sobre cama de aguacate premium.",
13        "price": "15.99",
```

... **http://3.20.57.154:3000/ops/menu/dishes**

POST **http://3.20.57.154:3000/ops/menu/dishes**

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {
2   "name": "Lomo de Cerdo",
3   "description": "Lomo en salsa de ciruelas",
4   "price": "12.5",
5   "categoryId": "cat_fuertes",
6   "imageUrl": null,
7   "isAvailable": true,
8   "availabilityReason": null
9 }
```

Body Cookies Headers 8 Test Results 200 OK • 264 ms

{} JSON Preview Visualize

```
1 {
2   "itemId": "ade90b83-9604-41d4-9970-8d302b8db928",
3   "name": "Lomo de Cerdo",
4   "description": "Lomo en salsa de ciruelas",
5   "price": "12.5",
6   "categoryId": "cat_fuertes",
7   "imageUrl": null,
8   "isAvailable": true,
9   "availabilityReason": null,
```




http://3.20.57.154:3000/ops/menu/dishes/c19b8822-db16-4fb1-b3a7-62635ba007a4

Save

DELETE

http://3.20.57.154:3000/ops/menu/dishes/c19b8822-db16-4fb1-b3a7-62635ba007a4

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 8 Test Results

200 OK • 260 ms •

{ } JSON Preview Visualize

```
1 {
2   "itemId": "c19b8822-db16-4fb1-b3a7-62635ba007a4",
3   "name": "Lomo de Cerdo",
4   "description": "Lomo en salsa de ciruelas",
5   "price": "12.5",
6   "categoryId": "cat_fuertes",
7   "imageUrl": null,
8   "isAvailable": true,
9   "availabilityReason": null,
10  "createdAt": "2026-06-10T03:01:15.743Z"
11 }
```

POST http: • POST http: • POST http: • POST http: • POST http: • POST http: • PUT New • DEL http: • POST ht

HTTP

http://3.20.57.154:3000/ops/customers/:customerId/preferences

PATCH

▼

http://3.20.57.154:3000/ops/customers/:customerId/preferences

☰ Docs

Params

● Authorization

Headers 9

Body ●

Scripts

Tests

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ▼

1 {

2 "preferences": {

3 "favoriteDishes": ["dish_001", "dish_002"],

4 "allergies": ["gluten", "lactose"],

5 "communicationPreferences": {

6 "email": true,

7 "sms": false

8 },

9 "loyaltyLevel": 2

10 }

11 }

Body

Cookies

Headers 8

Test Results

↺

{ } JSON ▼

▶ Preview

🖼️ Visualize ▼

1 {

2 "userId": "578a50a1-129c-4354-85e9-b09ac07b92c4",

3 "name": "Evelyn Villarreal",

4 "email": "ehvillarreal@espe.edu.com",

5 "phone": "+593 963208297",

6 "passwordHash": "scream it out loud",

7 "role": "customer",

8 "preferences": {

9 "allergies": ["gluten", "lactose"],

10 "communicationPreferences": {

11 "email": true,

12 "sms": false

13 },

14 "loyaltyLevel": 2

15 }

HTTP

http://3.20.57.154:3000/ops/menu/dishes/dish_001/ingredients

POST

▼

http://3.20.57.154:3000/ops/menu/dishes/dish_001/ingredients

☰ Docs

Params

Authorization

Headers 9

Body ●

Scripts

Tests

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ▼

1 {

2 "skuCode": "sku_salmon",

3 "quantityRequired": 200

4 }

Body

Cookies

Headers 8

Test Results

↺

500 Internal S

HTTP Save

POST http://3.20.57.154:3000/ops/auth/password/forgot

Docs Params Authorization Headers 9 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "email": "ehvillarreal@espe.edu.com"
3 }
4
```

Body Cookies Headers 8 Test Results 201 Created • 244 ms •

JSON Preview Visualize

```
1 {
2   "userId": "578a50a1-129c-4354-85e9-b09ac07b92c4",
3   "name": "Evelyn Villarreal",
4   "email": "ehvillarreal@espe.edu.com",
5   "phone": "+593 963208297",
6   "passwordHash": "scream it out loud",
7   "role": "customer",
8   "preferences": {},
9   "passwordResetToken": null,
10  "passwordResetExpires": "2023-05-10T18:00:00.000Z"
11 }
```

Global Vault Tools

GET http://3.20.57.154:3000/ops/auth/password/forgot POST http://3.20.57.154:3000/ops/auth/password/forgot POST http://3.20.57.154:3000/ops/auth/password/forgot POST http://3.20.57.154:3000/ops/auth/password/forgot POST http://3.20.57.154:3000/ops/auth/password/forgot POST http://3.20.57.154:3000/ops/auth/password/forgot PUT New DEL http://3.20.57.154:3000/ops/auth/password/forgot

HTTP Save

POST http://3.20.57.154:3000/ops/auth/password/forgot

Docs Params Authorization Headers 9 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "email": "ehvillarreal@espe.edu.ec"
3 }
4
```

Body Cookies Headers 6 Test Results

Raw Preview Visualize

```
1
```

HTTP **http://3.20.57.154:3000/ops/menu/dishes**

POST **http://3.20.57.154:3000/ops/menu/dishes**

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ☐

```
1 {
2   "name": "Ratatoille",
3   "price": 12.99,
4   "description": "Based in the movie",
5   "categoryId": "cat-001",
6   "imageUrl": "https://cdn.example.com/pizza.jpg",
7   "isAvailable": true
8 }
```

Body Cookies Headers 8 Test Results

{} JSON ☐ Preview Debug with AI ☐

```
1 {
2   ... "statusCode": 500,
3   "message": "Internal server error"
4 }
```

POST **http://3.20.57.154:3000/ops/auth/logout**

Send ☐

Docs Params Authorization **Headers** 8 Body Scripts Tests Settings

Cookies

Headers 7 hidden

Key	Value	Description	Bulk Edit	Presets	...
☒ x-user-id	578a50a1-129c-4354-85e9-b09ac07b92c4				
Key	Value	Description			

Body Cookies Headers 8 Test Results

201 Created • 262 ms • 282 B • ...

{} JSON ☐ Preview Visualize ☐

```
1 {
2   "count": 0
3 }
```

GET Get • POST http • POST http • POST http • GET http: • POST http • POST http • PUT New • DEL http: • POST http • + ▾

...  **http://3.20.57.154:3000/ops/auth/logout**  S

POST ▾ http://3.20.57.154:3000/ops/auth/logout

☰ Docs Params Authorization Headers 7 Body Scripts Tests Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 8 Test Results  201 Created • 258 r

{ } JSON ▾ ▶ Preview  Visualize ▾ 

```
1 {
2   |   "count": 3
3   }
```

...  **http://3.20.57.154:3000/ops/menu/dishes/dish_001**  Save ▾  Share 

PUT ▾ http://3.20.57.154:3000/ops/menu/dishes/dish_001  Send ▾ Cookies

☰ Docs Params Authorization Headers 9 Body • Scripts Tests Settings

raw ▾ JSON ▾  Schema Beautify

```
1 {
2   |   "name": "Tartar de Salmón y Aguacate",
3   |   "price": 15.99,
4   |   "isAvailable": true
5   }
```

Body ▾  200 OK • 329 ms • 619 B •  | ...

{ } JSON ▾ ▶ Preview  Visualize ▾  | ☰ 🔍 📄 🔗

```
1 {
2   |   "itemId": "dish_001",
3   |   "name": "Tartar de Salmón y Aguacate",
4   |   "description": "Salmón fresco marinado en cítricos sobre cama de aguacate
5   |                 premium.",
6   |   "price": "15.99",
7   |   "categoryId": "cat_entradas",
8   |   "imageUrl": "https://biconoirsgourmet/images/tartar.jpg",
9   |   "isAvailable": true,
```